



Fakultät Wirtschaft

Studiengang Wirtschaftsinformatik

**Elisa-AI
Artefakt
Pflichtenheft**

**Portfolio
im Modul
Integrationsseminar**

Im Rahmen der Prüfung zum Bachelor of Science (B. Sc.)

Mitglieder:

Daniel Sachadä

Patrick Molkenthin

Anas Haider-Mehmood

Lars Zieger

Ledejna Salihi

Sebastian Wolf

Stakeholder:

Dennis Schulmeister-Zimolong

Dozentin:

Prof. Dr. Katja Wengler

Abgabedatum:

26.01.2026

Inhaltsverzeichnis

Kapitel A: Zweck und Einordnung.....	1
1 Ausgangslage	1
2 Zielbild.....	1
Kapitel B: Zielsystem (Soll-Zustand).....	2
3 User Interface / User Experience (UI/UX).....	2
3.1 Farbkonzept	2
3.1.1 Farbtabelle (Universell)	2
3.1.2 Farbtabelle (Darkmode)	2
3.1.3 Farbtabelle (Lightmode)	3
3.2 Mockup-Screens und Beschreibung	3
3.2.1 User Dashboard.....	3
3.2.2 Chat Start Screen.....	4
3.2.3 Quiz Screen.....	4
3.2.4 Content View.....	5
3.3 Figma Link	5
3.4 Anlagen.....	5
4 Gamification-Konzept.....	6
4.1 Punkte und Skillpoints.....	6
4.2 Level	6
4.3 Streaks (Lernserien).....	6
4.4 Badges	6
4.5 Leaderboard (optional, anonymisiert).....	7
4.6 Lernspiele (PoC-Begrenzung)	7
4.7 (Side-)Quests und Challenges (optional).....	7
Kapitel C: System- und Architekturvorgaben	8
5 Analyse der Systemarchitektur	8
5.1 Das OpenBook Frontend	9

5.2	Der Openbook-Server	9
5.3	Das OpenBook Frontend Daten und KI-Infrastruktur	9
6	Bewertung des Architekturwechsels	10
Kapitel D: KI-Modell und Hosting		11
7	KI-Modell: Betriebsform und Begründung.....	11
8	Anbieterwahl: mittwald mStudio AI-Hosting	11
9	Modellstrategie.....	12
10	Hosting des Gesamtsystems.....	13
Kapitel E: Referenzsystem OpenBook		14
11	Beschreibung und Eigenschaften	14
12	Bedeutung des Berechtigungskonzepts	14
13	Einordnung im Projektkontext	15
14	Abgrenzung und Anpassung	15
Kapitel F: Datenmodell (Entwurfsstand).....		16
15	Zielbild und Grundprinzip	16
15.1	Login & Accounts: Identität, Rollen und Sitzungen	17
15.2	Chathistory: Normalisierter Verlauf über Chats und Messages.....	18
15.3	Content: Lernpfade, Module und dokumentbasierte Inhalte	19
15.4	Gamification: Definitionen, Ereignisprotokoll und aggregierter Zustan.....	20
Kapitel G: Projektbezogene Umsetzungsverantwortung.....		21
Kapitel H: Projektplan		23
16	Projektplanung	23
17	Meilensteine	23
17.1	Meilenstein 1 (M1): Technische Basis und Projektsetup	23
17.2	Meilenstein 2 (M2): Implementierung der Kernfunktionalitäten	24
17.3	Meilenstein 3 (M3): Gamification und Lernformate	24
17.4	Meilenstein 4 (M4): Integration und Systemstabilisierung	24
17.5	Meilenstein 5 (M5): Abschluss und Abnahme	24
18	Projektplan	25

18.1	Phasenübersicht und zeitliche Einordnung.....	26
Kapitel I:	Testkonzept.....	27
19	Pair Programming und Code-Review	27
20	Unit-Tests auf Code-Ebene	28
21	Meilensteinen Bezogene Funktionstests	28
22	Integrationstests	29
23	Abschließender Akzeptanztest	29
Kapitel J:	Lizenzen	31
Kapitel K:	Risiken und Gegenmaßnahmen.....	32
Kapitel L:	Traceability	33
	Literaturverzeichnis	35

Kapitel A: Zweck und Einordnung

1 Ausgangslage

Die traditionelle Lehr-Lern-Formate geraten durch die Verfügbarkeit generativer KI-Systeme (wie ChatGPT) unter Druck. Studierende nutzen diese Werkzeuge bereits, was zu Problemen wie unreflektiertem Gebrauch, Halluzinationen und der Gefahr des Deskillings (Verlust von Kompetenzen) führen kann.

Zudem sind Lehrende mit der Herausforderung konfrontiert, ihre PDF-basierten Vorlesungsskripte und Materialien in eine KI-gestützte Lernumgebung zu integrieren, ohne auf kommerzielle, datenschutzrechtlich bedenkliche oder proprietäre Lösungen zurückgreifen zu müssen.

2 Zielbild

Das Ziel ist die Entwicklung eines didaktisch fundierten, digitalen, autonomen und Open-Source-KI-Lernassistenten.

1. Didaktische Zielsetzung: Unterstützung des Lernprozesses durch didaktisch-methodische Hinweise, Lernplanung und die Berücksichtigung individueller Lernbedarfe.
2. Technische Zielsetzung: Bereitstellung einer 100% quelloffenen und freien Lizenz stehenden Anwendung, die auf den vorhandenen Materialien der Lehrenden aufbaut und eine personalisierte, interaktive Lernumgebung schafft.
3. Funktionales Ziel: Realisierung eines Proof of Concept (PoC), der eine funktionsfähige Webseite mit den Kernfunktionen Login, Lernpfad, Chat, Quiz und einem Spiel umfasst.

Kapitel B: Zielsystem (Soll-Zustand)

3 User Interface / User Experience (UI/UX)

Das UI/UX-Konzept basiert auf den bereitgestellten Mockups und Stilrichtlinien, die einen Darkmode mit futuristischem, klarem Design favorisieren (NFA08).

3.1 Farbkonzept

3.1.1 Farbtabelle (Universell)

Verwendung	Hexcode	Opacity
Warning Text	# A8A8A8	100%
Quiz Option 1	#FF5964	100%
Quiz Option 2	#35A7FF	100%
Quiz Option 3	#FFE74C	100%
Quiz Option 4	#6BF178	100%
Skillmatrixfläche	#6BF178	50%

3.1.2 Farbtabelle (Darkmode)

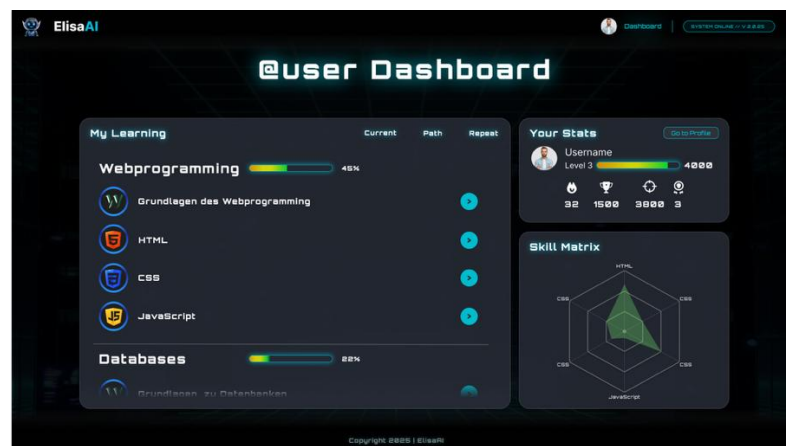
Verwendung	Hexcode	Opacity
Primär Text	#FFFFFF	100%
Textglow Effekt	#01BCCD	100%
Primär Hintergrund	#252A34	100%
Kachelhintergrund	#A8A8A8	10%

3.1.3 Farbtabelle (Lightmode)

Verwendung	Hexcode	Opacity
Primär Text	#252A34	100%
Sekundär Text	#01BCCD	100%
Primär Hintergrund	#FFFFFF	100%
Kachelhintergrund	#252A34	10%

3.2 Mockup-Screens und Beschreibung

3.2.1 User Dashboard

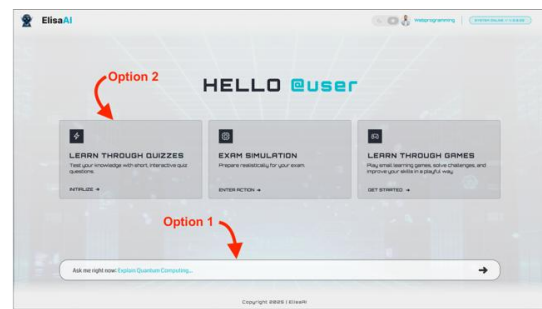
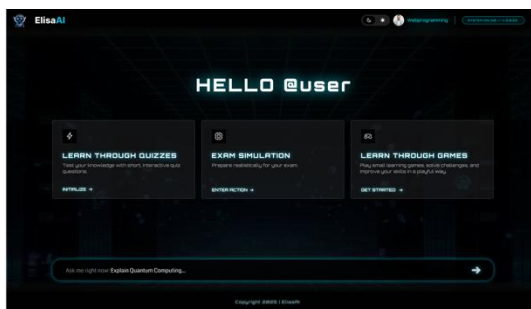


Zweck: Zentrale Anlaufstelle für den Lernenden.

Elemente:

4. **“My Learning” Card:** Zeigt den aktuell ausgewählten Lernpfad (z.B. “Webprogrammierung”) und den Lernfortschritt in Prozent an (FA06).
5. **Lerneinheiten:** Liste der Module (HTML, CSS, JavaScript, ...) mit Fortschrittsbalken und der Option “Repeat” (FA12).
6. **“Your Stats” Card:** Zeigt Gamification-Elemente wie Level, Streak-Anzahl und Skillpoints (FA09).
7. **Skill Matrix:** Visuelle Darstellung der erworbenen Skills/Kompetenzen (FA07).

3.2.2 Chat Start Screen



Zweck: Einstieg in die KI-Interaktion.

Option 1 (Direkt): Direkte Frage an ElisaAI zur Lernveranstaltung (reaktiver Chat, FA08).

Option 2 (QuickCards): Auswahl von Aktivitäten wie “Learn Through Quizzes”, “Exam Simulation” oder “Learn Through Games” (FA05, FA10).

3.2.3 Quiz Screen

ElisaSplit



ElisaFloat



ElisaIcon



Zweck: Durchführung von KI-generierten Quizzes.

Varianten:

- **ElisaSplit:** Quiz links, Chat rechts. Ermöglicht es dem Nutzer, während des Quiz Fragen an ElisaAI zu stellen, ohne die Aufgabe zu verlassen.
- **ElisaFloat:** Chat als Floating-Element über dem Quiz.
- **ElisaIcon:** Chat ist ausgeblendet und nur über ein Icon zugänglich.

Funktion: ElisaAI erstellt automatisch ein Quiz, das an die Modulinhalt angepasst ist. Direkte Rückmeldung nach jeder Antwort.

3.2.4 Content View

ElisaSplit



ElisaFloat



ElisaIcon



Zweck: Bereitstellung und Navigation von Kurs-/Vorlesungsinhalten (z. B. Kapitel, Themen, Einheiten) mit der Option, Inhalte direkt im Kontext durch ElisaAI erklären zu lassen.

Varianten:

- **ElisaSplit:** Content links, Chat rechts. Ermöglicht es dem Nutzer, fragen zu den Vorlesungsinhalten an ElisaAI zu stellen, ohne die Aufgabe zu verlassen.
- **ElisaFloat:** Chat als Floating-Element über dem Content.
- **ElisaIcon:** Chat ist ausgeblendet und nur über ein Icon zugänglich.

Funktion: Der Content Screen zeigt das ausgewählte Lernmaterial im Hauptbereich, während der integrierte Chat kontextbezogene Fragen und Erklärungen ermöglicht, ohne den Inhalt zu verlassen.

3.3 Figma Link

Für die Erstellung der Mockups wurde das Design-Tool Figma verwendet. Die zugehörigen Figma-Ressourcen sind unter dem folgenden Link abrufbar:

<https://www.figma.com/design/8UJyGd75P9l51JO2eVAWox/ElisaAI?node-id=120-220&t=dhQSIGd3hBuCr1u0-1>

3.4 Anlagen

Die Abbildungen sind in der Anlage „Beigabe_2-mockups“ sowohl als PDF- als auch als JPG-Dateien enthalten.

4 Gamification-Konzept

Zur Steigerung der Lernmotivation ergänzt ElisaAI die fachlichen Kernfunktionen (Chat, Content-View, Lernpfad) um eine leichte Gamification-Ebene (FA09). Ziel ist nicht „Spiel um des Spiels willen“, sondern ein sichtbares, positives Feedback auf Lernaktivität und Lernfortschritt. Die Mechaniken sind bewusst einfach gehalten, damit sie transparent bleiben und im Proof of Concept robust umgesetzt werden können.

4.1 Punkte und Skillpoints

Als Basis dient ein Punktesystem: Für korrekt gelöste Aufgaben (z. B. Quizfragen) werden Punkte vergeben (Richtwert: 10 Punkte pro richtiger Antwort). Ergänzend dazu werden Skillpoints als kompetenzbezogene Kennzahl geführt. Skillpoints sind nicht an einen einzelnen Raum gebunden, sondern raum- bzw. modulübergreifend (FA07/FA09), damit Fortschritt langfristig sichtbar bleibt und nicht bei jedem Themenwechsel „neu startet“.

4.2 Level

Auf Basis des gesammelten Punktekontos steigt der Nutzer in Levels auf (z. B. Level 1–10). Levels dienen primär der Orientierung und Visualisierung im Dashboard („Your Stats“) und sind keine Zugangsbeschränkung für Inhalte.

4.3 Streaks (Lernserien)

Um regelmäßiges Lernen zu fördern, wird eine Streak geführt: Jeder Tag mit aktiver Lerninteraktion erhöht die Serie. Optional kann eine aktive Streak einen Punktemultiplikator erzeugen (z. B. 3-Tage-Streak: $10 \text{ Punkte} \times 1,3$). Diese Logik ist als Motivator gedacht und wird so umgesetzt, dass sie leicht verständlich bleibt.

4.4 Badges

Badges sind Meilenstein-Auszeichnungen (FA15), z. B. für das Abschließen eines Raums oder das Erreichen eines Levels. Sie haben vor allem Sammler- und Anerkennungscharakter und unterstützen die Progression, ohne die fachliche Bewertung zu ersetzen.

4.5 Leaderboard (optional, anonymisiert)

Ein Leaderboard pro Raum (FA14) ist optional und wird nur umgesetzt, wenn es didaktisch sinnvoll und datenschutzkonform realisierbar ist. Dabei gilt: keine Klarnamen, sondern pseudonymisierte Anzeige. Das Feature wird als „nice to have“ betrachtet, da Ranglisten demotivierend wirken können.

4.6 Lernspiele (PoC-Begrenzung)

Im Proof of Concept werden maximal **zwei** spielerische Formate umgesetzt (FA05/FA10), um den Umfang kontrollierbar zu halten:

1. **Quiz** als zentrales Interaktionselement mit direkter Rückmeldung und Punktevergabe.
2. Optional ein einfaches Spiel wie **Memory** oder ein vergleichbares Matching-Format, das sich gut mit Lerninhalten verbinden lässt.

4.7 (Side-)Quests und Challenges (optional)

Als freiwillige Zusatzaufgaben (FA16) können Quests als kurzfristige Motivatoren dienen, vorzugsweise vorlesungs- bzw. raumspezifisch (z. B. Wochen-Challenges). Beispiele sind „Sammle 300 Punkte“ oder „Schließe 3 Räume ab“. Als Belohnung kommen Extrapunkte oder ein „Streak-Freeze“ in Frage, um einzelne Ausfalltage abzufedern. Die Umsetzung ist optional und abhängig vom PoC-Fortschritt.

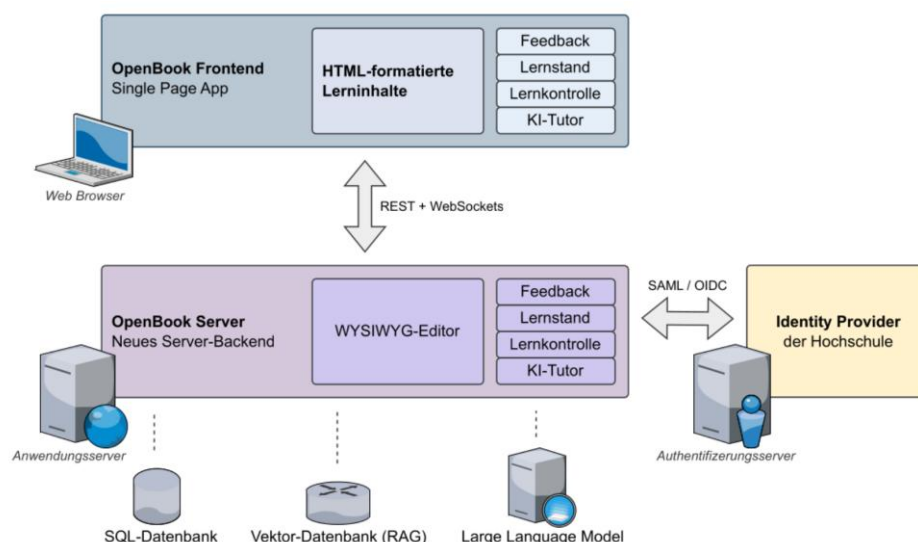
Kapitel C: System- und Architekturvorgaben

Das Projekt „Elisa-AI“ integriert einen intelligenten KI-Lerntutor in das OpenBook-Ökosystem. Die vorliegende Architekturzeichnung beschreibt ein System, das auf Skalierbarkeit und Echtzeitinteraktion optimiert ist. Im Gegensatz zu klassischen, oft schwerfälligen 4-Layer-Architekturen setzt dieser Entwurf auf eine klare Trennung zwischen einem reaktiven Client, einem hochspezialisierten Logik-Server und modernen Datendiensten

5 Analyse der Systemarchitektur

Im Rahmen der initialen Konzeptionsphase wurde zunächst eine klassische 4-Schichten-Architektur (Presentation, Business Logic, Data Access und Database) in Betracht gezogen. Während der detaillierten Anforderungsanalyse für Elisa-AI stellte sich jedoch heraus, dass die hohen Latenzanforderungen von Large Language Models (LLMs) sowie die notwendige Echtzeit-Kommunikation (Streaming) eine flachere und spezialisiertere Struktur erfordern.

Die im Folgenden dargestellte Architektur markiert den Übergang von einem starren, schichtgebundenen Modell hin zu einer modularen, serviceorientierten Struktur. Diese Reduktion von Komplexität zugunsten von Performance ermöglicht eine direkte Anbindung der KI-Komponenten bei gleichzeitiger Wahrung der Sicherheit und Skalierbarkeit.



Die in Abbildung 1 visualisierte Architektur gliedert sich in drei funktionale Hauptbereiche: Die Client-Ebene (Frontend), die Orchestrierungsebene (Backend) und die Daten- bzw. KI-Ebene (Infrastruktur).

5.1 OpenBook Frontend

Das Frontend ist als Single Page Application (SPA) konzipiert und wird mit dem Framework Svelte realisiert. Svelte bietet durch seinen kompilierbasierten Ansatz einen minimalen Footprint und eine hohe Ausführungsgeschwindigkeit, was für die reaktive Darstellung der KI-Tutor-Antworten essenziell ist.

- **Funktionsumfang:** Neben der Darstellung HTML-formatierter Lerninhalte integriert das Frontend Module für direktes Feedback, die Lernstandsanzeige sowie die Schnittstelle zum KI-Tutor.
- **Schnittstellen:** Die Kommunikation erfolgt hybrid über REST für zustandslose Operationen und WebSockets für die bidirektionale Echtzeit-Interaktion mit dem KI-System.

5.2 Openbook-Server

Das zentrale Bindeglied der Architektur bildet der Anwendungsserver, der auf Python basiert. Die Wahl von Python als Backend-Sprache ist strategisch begründet durch die nahtlose Integration führender KI-Frameworks und Bibliotheken (wie LangChain oder FastAPI).

- **Zentrale Dienste:** Der Server beherbergt den WYSIWYG-Editor für Lehrende sowie die Geschäftslogik für die Lernkontrolle.
- **KI-Steuerung:** Er fungiert als Gateway zum LLM und steuert den Informationsfluss zwischen der Benutzeranfrage und den Hintergrundsystemen.
- **Authentifizierung:** Über eine standardisierte Schnittstelle (SAML / OIDC) wird die Identität der Nutzer gegen den hochschuleigenen Identity Provider (IdP) validiert, was ein sicheres Single-Sign-On (SSO) ermöglicht.

5.3 Das OpenBook Frontend Daten und KI-Infrastruktur

Die unterste Ebene stellt die notwendigen Ressourcen für die Datenhaltung und die generative Intelligenz bereit:

- **SQL-Datenbank:** Speicherung persistenter, strukturierter Daten wie Nutzerprofile und Kursmetadaten.

- Vektor-Datenbank (RAG): Das Herzstück für die fachliche Präzision von Elisa-AI. Hier werden Lerninhalte als Vektoren (Embeddings) gespeichert. Mittels Retrieval Augmented Generation (RAG) extrahiert das System relevante Kontexte, um die Antworten des KI-Tutors an das spezifische Lehrmaterial zu binden.
- Large Language Model (LLM): Die generative Komponente, die über den Python-Server angesteuert wird, um natürliche Sprache zu verarbeiten und Antworten zu formulieren.

6 Bewertung des Architekturwechsels

Der Vorteil gegenüber der ursprünglich geplanten 4-Layer-Architektur liegt in der horizontalen Durchlässigkeit. In einem klassischen Schichtenmodell müsste jede KI-Anfrage vier Abstraktionsschichten durchlaufen, was die Antwortzeiten (Time to First Token) signifikant erhöhen würde.

Die neue Architektur ermöglicht dem Python-Backend eine direkte, parallele Abfrage von SQL- und Vektordaten. Durch die Entkopplung der KI-Dienste von der restlichen Business-Logik ist Elisa-AI zudem hochgradig wartbar: Das LLM oder die Vektor-Datenbank können aktualisiert werden, ohne die Stabilität des Frontends oder der Kern-Datenbank zu gefährden.

Kapitel D: KI-Modell und Hosting

7 KI-Modell: Betriebsform und Begründung

Für ElisaAI wird im PoC eine **API-basierte Nutzung** des Sprachmodells empfohlen. Dadurch entfallen eigener GPU-Betrieb und der größte Teil des infrastrukturellen Aufwands (Skalierung, Modellbetrieb, Updates der Modellumgebung). Self-Hosting bleibt als Option für später bestehen, ist im PoC aber nicht vorgesehen, da es zusätzliche Engineering- und Betriebskosten verursacht (Provisionierung, Monitoring, Ausfallsicherheit).

Entscheidungskriterien im Projekt:

- **Planbare Kosten** (monatliche Fixkosten statt schwankender Tokenrechnungen)
- **Kontextlänge** (Material + Chatverlauf in einer Anfrage verarbeiten)
- **Fähigkeiten** (Tool-Calling, optional Vision/Speech-to-Text)
- **Datenschutz/Standort** (EU/DE-Betrieb)
- **Technische Integration** (Streaming + strukturierte Ausgaben für Quiz)

8 Anbieterwahl: mittwald mStudio AI-Hosting

Als Anbieter wird **mittwald mStudio AI-Hosting** empfohlen. Die Anbindung würde über eine **OpenAI-kompatible API** erfolgen.

Tarife und Limits (netto, zzgl. USt.)

- **Starter:** 9 €/Monat, 5 Mio. Tokens/Monat, 30 Requests/Minute
- **Pro:** 39 €/Monat, 75 Mio. Tokens/Monat, 60 Requests/Minute
- **Business:** 149 €/Monat, 300 Mio. Tokens/Monat, 150 Requests/Minute
- **Dedicated (Enterprise):** 999 €/Monat, dedizierte GPU-Ressourcen (RTX PRO 6000), sehr hohe Token kontingente

Starttarif für den PoC: Starter, da Token Puffer und Rate-Limit im Verhältnis zu den Kosten einen stabilen Betrieb ausreichen würden.

9 Modellstrategie

Für ElisaAI wäre eine einfache Modellstrategie sinnvoll: **ein Standardmodell** für den Regelfall und **ein stärkeres Modell** als Fallback, falls Qualität/Reasoning nicht ausreicht. Zusätzlich wird ein Embedding-Modell für RAG vorgesehen.

Verfügbare Modelle im mittwald AI-Hosting (Auszug, relevant für ElisaAI):

- **gpt-oss-120b**
Open-Source-Modell von OpenAI, veröffentlicht unter Apache 2.0-Lizenz. Starkes reasoning und agentische Fähigkeiten bei effizientem Ressourcenverbrauch.
- **Devstral-Small-2-24B-Instruct-2512**
Coding-Modell mit 24 Milliarden Parametern, optimiert für moderne Entwicklungs-Workflows. Perfekt für Code-Generierung, Refactoring, Debugging und agentische Programmieraufgaben.
- **Ministral-3-14B-Instruct-2512**
Europäisches Open-Source-Modell mit 14 Milliarden Parametern, optimiert für hochwertige Text-, Chat- und Vision-Anwendungen in Webprojekten.
- **Qwen3-Embedding-8B**
Embedding-Modell zur Erzeugung semantischer Vektorrepräsentationen aus Text. Es bildet die Grundlage für RAG, indem Inhalte in einer Vektor-Datenbank gespeichert und für eine semantische Suche genutzt werden, deren Treffer anschließend in die KI-Antworten integriert werden können.
- **Whisper-Large-V3-Turbo**
Leistungsstarkes Speech-to-Text-Modell für schnelle und präzise Transkription in zahlreichen Sprachen. Ideal, um Meetings, Podcasts, Webinare oder Support-Anrufe in Text umzuwandeln und diese weiterzuverarbeiten.

10 Hosting des Gesamtsystems

Das LLM kann über **mittwald AI-Hosting** bezogen werden. Die übrigen Systembestandteile (Frontend, Backend, Datenhaltung und RAG-Komponenten) können unabhängig davon betrieben werden. Für ElisaAI wäre es effizient, auch diese Komponenten in einer **extern betriebenen Hosting-Umgebung** zu betreiben – idealerweise beim selben Anbieter bzw. in einer Umgebung mit **Hosting in Deutschland/EU**. Dadurch würden Betriebsaufwände reduziert (Bereitstellung, Wartung, Updates, Backups, Verfügbarkeit), ohne dass das Projekt eigene Infrastruktur aufzubauen und zu administrieren.

Aus Datenschutzsicht ist das besonders relevant, weil ElisaAI personenbezogene Daten verarbeiten kann (z. B. Accounts, Lernfortschritt, Chat-Verläufe, hochgeladene Materialien). Ein Hosting bei einem Anbieter mit DSGVO-konformer Ausrichtung und klaren Vertragsgrundlagen (z. B. Auftragsverarbeitung) vereinfacht die Umsetzung von Anforderungen wie Datenminimierung, Zugriffskontrolle, sichere Speicherung und Nachvollziehbarkeit. Zusätzlich ließe sich durch die Wahl eines deutschen/EU-Hostings das Risiko unerwünschter Drittstaatentransfers reduzieren.

Kapitel E: Referenzsystem OpenBook

Im Rahmen der technischen Konzeption des Projekts ElisaAI wurde entschieden, sich bei der Ausgestaltung grundlegender Systemkomponenten an einer bereits existierenden Anwendung mit dem Namen „OpenBook“ zu orientieren. OpenBook ist eine eigenständige Webanwendung, die im Projektkontext nicht als fertige Endlösung, sondern als Referenz- und Ausgangsbasis für zentrale technische Konzepte herangezogen wird.

Der Einsatz von OpenBook verfolgt das Ziel, den Entwicklungsaufwand für grundlegende Infrastrukturkomponenten zu reduzieren und bewährte architektonische Lösungen zu nutzen. Dadurch soll es dem Projektteam ermöglicht werden, sich stärker auf die fachlichen und inhaltlichen Schwerpunkte des Projekts zu konzentrieren, insbesondere auf KI-gestützte Lernmechanismen, Gamification-Elemente und die didaktische Ausgestaltung der Lernlogik.

11 Beschreibung und Eigenschaften

OpenBook basiert auf einer etablierten Backend-Architektur unter Verwendung des Django-Frameworks und bringt bereits mehrere grundlegende Funktionen mit, die für vergleichbare Webanwendungen typisch sind. Dazu zählen unter anderem:

- eine strukturierte und erprobte Backend-Grundarchitektur,
- eine integrierte Benutzer- und Rollenverwaltung,
- ein administratives Interface zur Systemverwaltung,
- eine REST-basierte Programmierschnittstelle (API),
- sowie automatisierte Unit-Tests zur Qualitätssicherung.

Diese vorhandenen Komponenten bilden eine solide Grundlage, um technische Standardanforderungen nicht neu implementieren zu müssen, sondern auf bestehenden Konzepten aufzubauen.

12 Bedeutung des Berechtigungskonzepts

Ein wesentlicher Grund für die Nutzung von OpenBook liegt im bereits implementierten **Berechtigungs- und Rollenmodell**. Dieses Modell ermöglicht eine klare und nachvollziehbare Steuerung von Zugriffsrechten und dient als konzeptionelle Vorlage für das Berechtigungssystem von ElisaAI.

Das Berechtigungskonzept von OpenBook basiert auf einem rollenbasierten Ansatz, bei dem:

- Benutzer bestimmten Rollen zugeordnet werden,
- Rollen definierte Rechte besitzen,
- Zugriffsprüfungen zentral auf Backend- und API-Ebene erfolgen,
- administrative Aufgaben über ein dediziertes Admin-Interface verwaltet werden.

Für ElisaAI ist ein solches Modell besonders relevant, da unterschiedliche Nutzergruppen (z. B. Lernende, Lehrende, Administratoren) vorgesehen sind, die jeweils unterschiedliche Rechte und Zugriffsmöglichkeiten benötigen.

13 Einordnung im Projektkontext

OpenBook wurde im Projekt bewusst als **Unterstützung und Beschleuniger** eingeplant. Ziel ist es nicht, Funktionalitäten oder fachliche Inhalte direkt zu übernehmen, sondern vor allem **technische Basisstrukturen** wie das Berechtigungs- und Rollenmodell als Ausgangspunkt zu nutzen.

Durch diese Vorgehensweise kann der Entwicklungsfokus gezielt verlagert werden:

- weg von der Implementierung grundlegender Infrastruktur,
- hin zur Ausarbeitung projektspezifischer Funktionen,
- insbesondere im Bereich KI-Logik, Gamification und Lernprozesse.

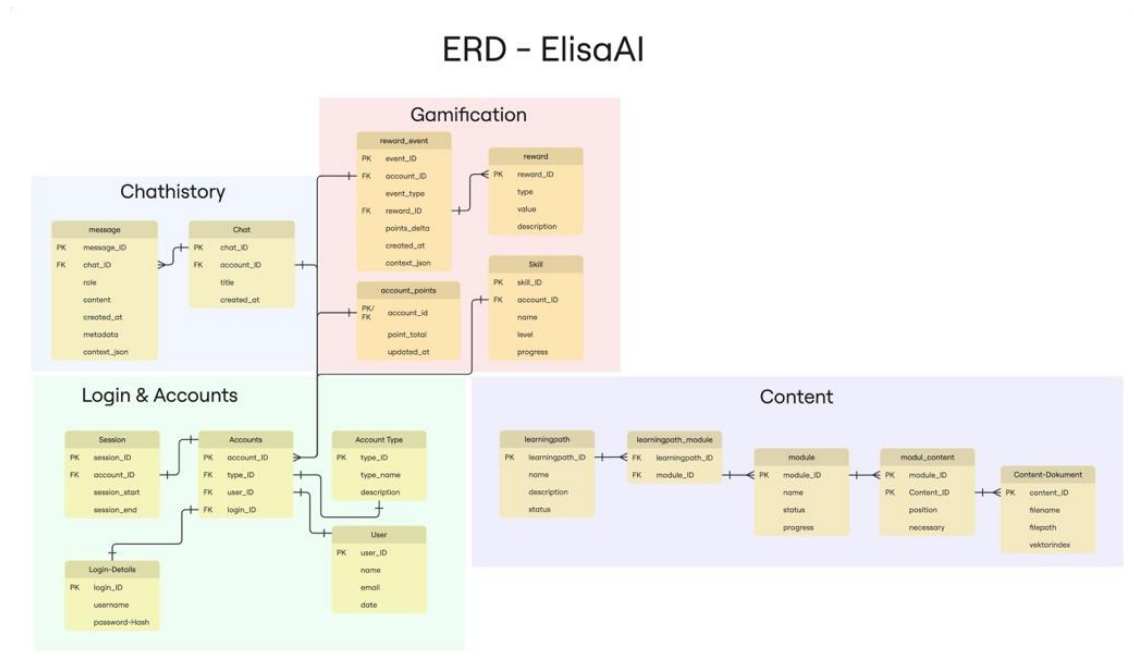
OpenBook stellt somit ein Mittel dar, um Entwicklungsrisiken zu minimieren und die Komplexität der technischen Basis zu reduzieren.

14 Abgrenzung und Anpassung

Die Nutzung von OpenBook erfolgt nicht unverändert oder vollständig. Die im Projekt umgesetzten Konzepte werden bedarfsgerecht angepasst und erweitert, um den funktionalen und nicht-funktionalen Anforderungen von ElisaAI zu entsprechen.

Kapitel F: Datenmodell (Entwurfsstand)

15 Zielbild und Grundprinzip

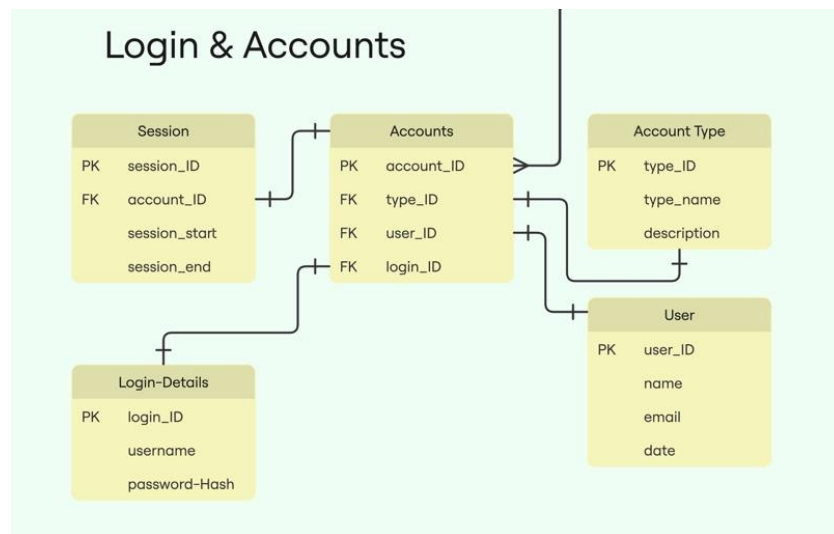


Die Datenbank strukturiert ElisaAI in klar getrennte Funktionsbereiche (Login & Accounts, Chathistory, Content, Gamification). Die Verknüpfung der Bereiche erfolgt überwiegend über Fremdschlüsselbeziehungen, wodurch referenzielle Integrität und nachvollziehbare Datenflüsse gewährleistet werden. Als zentraler Identifikator fungiert account, an den die meisten nutzungs- und lernbezogenen Informationen angebunden sind.

Hinweise:

- Reifegrad: Der Entwurf bildet die Zielrichtung ab, ist jedoch nicht final.
- Die Abbildungen sind in der Anlage „Beigabe_3-datendiagramme“ sowohl als PDF- als auch als JPG-Dateien enthalten.

15.1 Login & Accounts: Identität, Rollen und Sitzungen

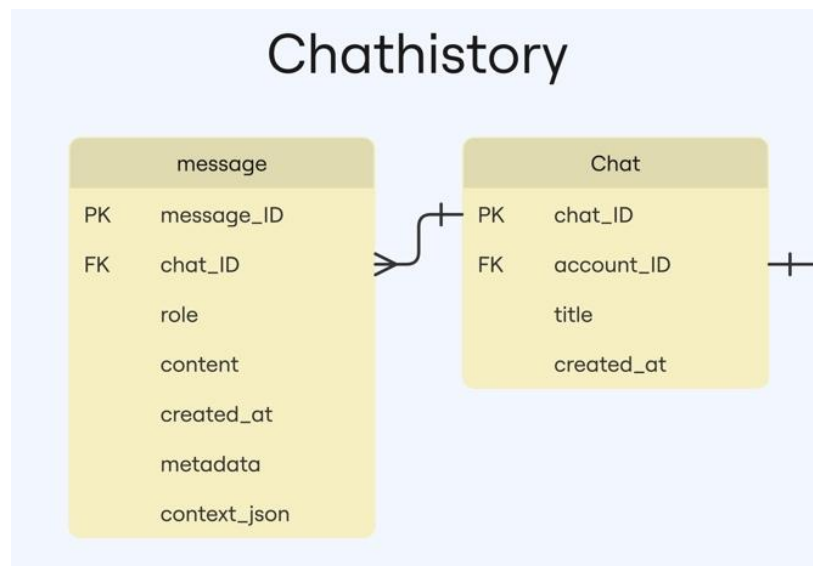


Der Bereich Login & Accounts trennt Stammdaten, Zugangsdatensätze und Nutzungssitzungen:

- `login_details` enthält die Anmeldeinformationen (z. B. `username`, `password_hash`) als technische Grundlage der Authentifizierung.
- `user` hält die personenbezogenen Profildaten (z. B. Name, E-Mail).
- `account` bildet die fachliche Identität innerhalb des Systems ab und verknüpft:
 - `user_id` → Profilzuordnung,
 - `login_id` → Login-Datensatz,
 - `type_id` → Account-Typ (Rolle/Variante) über `account_type`.
- `session` protokolliert Sitzungen je Account (`session_start`, `session_end`) und bildet damit eine 1:n-Beziehung ab: ein Account, viele Sessions

Damit ist klar getrennt, wer der Nutzer ist (`user`), wie er sich authentifiziert (`login_details`) und in welcher Systemrolle/Instanz er agiert (`account`).

15.2 Chathistory: Normalisierter Verlauf über Chats und Messages



Der Chatverlauf ist in Kopf- und Detaildaten normalisiert:

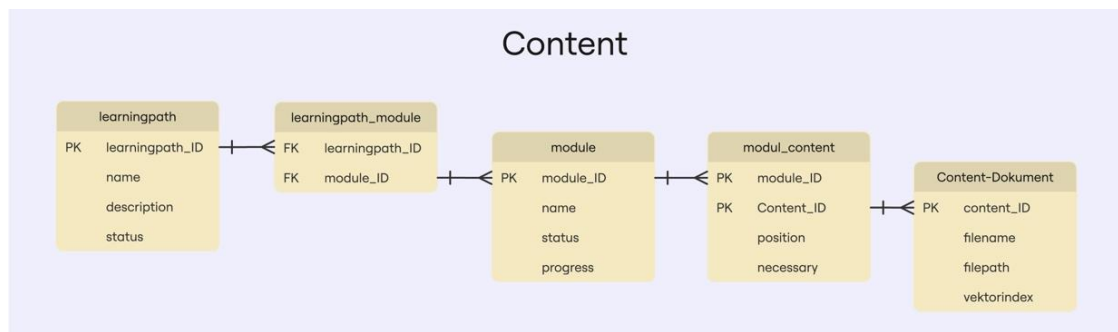
chat repräsentiert eine Chat-Sitzung (z. B. Titel, Erstellzeitpunkt) und ist über `account_id` einem Account zugeordnet: ein Account, viele Chats.

message speichert jede Nachricht als eigenständigen Datensatz und verweist via `chat_id` auf den zugehörigen Chat: ein Chat, viele Messages.

Felder wie `role` (z. B. user/assistant), `metadata` und `context_json` ermöglichen die Ablage technischer Zusatzinformationen (z. B. Modellkonfiguration, Retrieval-Kontext) ohne die Kernstruktur aufzubrechen.

Die wesentliche Verbindung lautet damit: `account → chat → message`, wodurch sich Verläufe performant filtern, sortieren und auswerten lassen.

15.3 Content: Lernpfade, Module und dokumentbasierte Inhalte

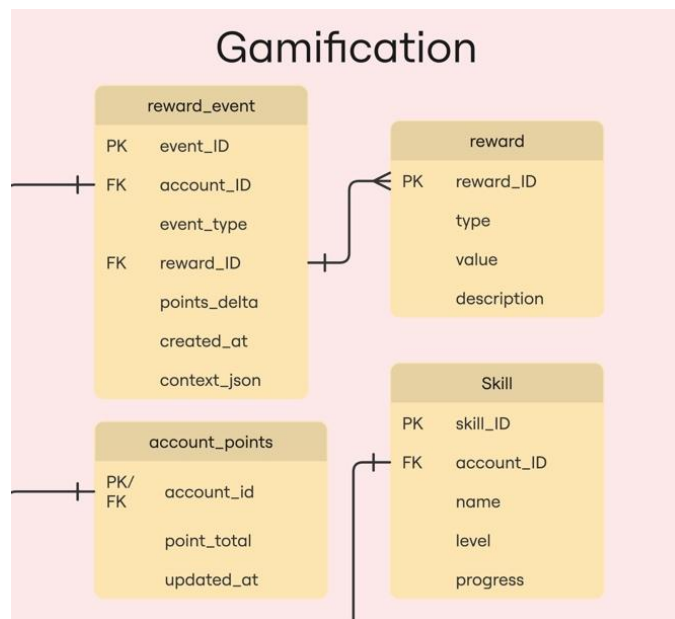


Die Lernstruktur ist hierarchisch aufgebaut und an mehreren Stellen über n:m-Zuordnungen flexibel gehalten:

learningpath	beschreibt Lernpfade (Name, Beschreibung, Status)
learningpath_module	ist eine Zuordnungstabelle für die n:m-Beziehung: ein Lernpfad enthält viele Module und ein Modul kann in mehreren Lernpfaden wiederverwendet werden.
module	beschreibt Module als didaktische Einheiten (Name, Status; derzeit auch progress als Entwurfssfeld).
content_document	verwaltet Inhaltsdokumente (Dateiname, Pfad) und eine Referenz auf einen Vektorindex (vektorindex) zur späteren Retrieval-/RAG-Nutzung.
modul_content	ordnet Inhalte Modulen zu (n:m) und erweitert die Beziehung um didaktische Attribute:
position	zur Reihenfolge,
necessary	zur Kennzeichnung von Pflichtinhalten.

Die zentrale Verbindung im Content-Bereich lautet: learningpath ↔ module ↔ content_document, wobei die „↔“-Beziehungen über Zwischentabellen umgesetzt sind.

15.4 Gamification: Definitionen, Ereignisprotokoll und aggregierter Zustand



Gamification ist als Kombination aus Definition und Event-Historie angelegt:

- reward definiert Belohnungen bzw. Gamification-Objekte (z. B. Typ, Wert, Beschreibung).
- reward_event protokolliert konkrete Ereignisse pro Account (z. B. Punktevergabe, Badge-Erhalt) mit:
 - account_id,
 - reward_id,
 - event_type,
 - points_delta,
 - created_at
- context_json für Kontext (z. B. Modulbezug/Begründung).
- account_points hält als aggregierter Zustand den aktuellen Punktestand je Account (Cache/State für schnelle Leaderboards/Profilansichten).

Damit entsteht ein nachvollziehbarer Verlauf (Event Trail) bei gleichzeitiger Performanz über den aggregierten Punktestand.

Kapitel G: Projektbezogene Umsetzungsverantwortung

Zur strukturierten Umsetzung des Proof of Concept werden für zentrale System- und Codebereiche klare Verantwortlichkeiten definiert. Ziel ist es, Zuständigkeiten transparent zu machen, die Koordination innerhalb des Teams zu erleichtern und eine nachvollziehbare Aufteilung der Entwicklungsarbeit sicherzustellen.

Bereich	Inhaltliche Verantwortung	Primär verantwortlich
Frontend – Basis & Navigation	Grundstruktur des Frontends, Routing, Layout, UI-Grundkomponenten	Lars, Ledejna
Backend – Lernpfade & Gamification	Dashboard, Quiz-Darstellung, Gamification-Elemente	Lars, Ledejna
Backend – Basis & API & Datenmodell	Grundarchitektur, API-Endpunkte, Validierung, Schnittstellen, Datenmodelle, Speicherung von Nutzer- und Lerndaten	Anas, Patrick
Backend – KI-Logik, Chat	Chat-Logik, KI-Integration, Prompt-Steuerung	Sebastian, Daniel
Tests & Qualitätssicherung	Unit-Tests, meilensteinbezogene Funktionstests, Testkoordination	teamübergreifend
Integration & Code-Review	Zusammenführung der Komponenten, Reviews, Merge-Strategien	teamübergreifend

Die Zuordnung der Verantwortlichkeiten erfolgt auf Ebene von Komponenten und Funktionsbereichen, nicht auf Ebene einzelner Dateien oder Klassen. Die jeweils benannten Personen tragen die primäre Verantwortung für Konzeption, Implementierung und Pflege des zugeordneten Bereichs.

Die Umsetzung erfolgt grundsätzlich im Pair Programming, sodass Entwicklungsaufgaben jeweils von zwei Teammitgliedern gemeinsam bearbeitet werden. Dabei übernimmt eine Person die primäre fachliche Verantwortung für den jeweiligen Bereich, während die zweite Person unterstützend, prüfend und reflektierend eingebunden ist. Dieses Vorgehen

dient der Qualitätssicherung, dem Wissensaustausch innerhalb des Teams sowie der Reduktion von Fehlern.

Ergänzend zum Pair Programming werden regelmäßige Code-Reviews und Abstimmungen durchgeführt, um eine konsistente Umsetzung und eine saubere Integration der einzelnen Komponenten sicherzustellen.

Kapitel H: Projektplan

16 Projektplanung

Dieses Kapitel beschreibt die Projektplanung für die Umsetzung der im Integrationsseminar entwickelten Artefakte. Auf Grundlage des finalen Pflichtenhefts wird der geplante Ablauf der Realisierungsphase dargestellt und in Meilensteine, Umsetzungsphasen und Verantwortlichkeiten gegliedert.

Der Projektplan dient als strukturierender Rahmen für die Umsetzung des Proof of Concept. Er definiert zentrale Meilensteine, ordnet die wesentlichen Entwicklungsaufgaben zeitlich ein und weist Verantwortlichkeiten für relevante System- und Codebereiche zu. Die Planung erfolgt bewusst auf einem groben Detaillierungsgrad und bildet die Grundlage für die nachfolgende Implementierungsphase.

17 Meilensteine

Zur Strukturierung der Umsetzungsphase wird das Projekt in klar definierte Meilensteine unterteilt. Die Meilensteine dienen der zeitlichen Orientierung, der Fortschrittskontrolle sowie der inhaltlichen Abgrenzung zentraler Entwicklungsschritte. Jeder Meilenstein beschreibt einen überprüfbaren Zustand des Systems und markiert den Abschluss eines wesentlichen Projektabschnitts.

17.1 Meilenstein 1 (M1): Technische Basis und Projektsetup

Ziel dieses Meilensteins ist die Herstellung einer stabilen technischen Ausgangsbasis für die weitere Entwicklung. Dazu gehören die Einrichtung der Entwicklungsumgebung, die Grundstruktur des Frontend- und Backend-Systems sowie die Definition zentraler technischer Schnittstellen. Am Ende dieses Meilensteins liegt ein lauffähiges Grundsystem vor, das als Basis für die Implementierung der Fachfunktionen dient.

Ergebnis:

Lauffähige Projektstruktur mit angebundenem Frontend und Backend sowie initialem Systemgerüst.

17.2 Meilenstein 2 (M2): Implementierung der Kernfunktionalitäten

In diesem Meilenstein werden die zentralen Funktionen des Proof of Concept umgesetzt. Dazu zählen insbesondere die grundlegenden Nutzerinteraktionen, die Chat-Funktionalität sowie die Einbindung von Lerninhalten gemäß dem im Pflichtenheft beschriebenen Soll-Konzept. Der Fokus liegt auf der funktionalen Abbildung der Hauptanwendungsfälle.

Ergebnis:

Nutzbare Kernfunktionen mit durchgängigen Nutzerinteraktionen.

17.3 Meilenstein 3 (M3): Gamification und Lernformate

Aufbauend auf den Kernfunktionalitäten werden in diesem Meilenstein die vorgesehenen Gamification-Elemente sowie die Lernformate umgesetzt. Dazu zählen insbesondere Quiz-Funktionalitäten sowie weitere spielerische Elemente im Rahmen des Proof of Concept. Ziel ist es, den Lernprozess um motivationale Komponenten zu ergänzen und technisch in das Gesamtsystem zu integrieren.

Ergebnis:

Integrierte Gamification-Mechaniken und funktionsfähige Lernformate.

17.4 Meilenstein 4 (M4): Integration und Systemstabilisierung

Dieser Meilenstein dient der Zusammenführung aller implementierten Komponenten zu einem konsistenten Gesamtsystem. Neben der technischen Integration stehen die Behebung von Fehlern, kleinere Optimierungen sowie die Überprüfung des Systemverhaltens im Fokus. Ziel ist ein stabiler, demonstrierbarer Systemstand.

Ergebnis:

Stabiles, integriertes Gesamtsystem des Proof of Concept.

17.5 Meilenstein 5 (M5): Abschluss und Abnahme

Im abschließenden Meilenstein erfolgt die Vorbereitung der finalen Abgabe. Dazu gehören die abschließende Überprüfung der Umsetzung, die Dokumentation des Projekts sowie die Vorbereitung einer Demonstration des Systems. Der Meilenstein markiert den formalen Abschluss der Umsetzungsphase.

Ergebnis:

Abnahmefähiger Proof of Concept inklusive Dokumentation und Präsentationsgrundlage.

Die Meilensteine orientieren sich am im Pflichtenheft beschriebenen Soll-Konzept und bilden den schrittweisen Aufbau des Proof of Concept ab. Dabei wird die geplante Umsetzung der im Lastenheft formulierten Anforderungen in logisch aufeinander aufbauende Projektabschnitte überführt.

18 Projektplan

Der Projektplan beschreibt die zeitliche Strukturierung der Umsetzungsphase und ordnet die Entwicklungsaktivitäten in aufeinander aufbauende Phasen ein. Die Planung erfolgt bewusst als grobe Zeitplanung und dient der Orientierung sowie der Koordination der Projektarbeit. Abweichungen von den vorgesehenen Zeiträumen sind projektbedingt möglich und werden im Rahmen der iterativen Entwicklung berücksichtigt.

Die Umsetzungsphase gliedert sich in mehrere Phasen, die jeweils einem oder mehreren Meilensteinen zugeordnet sind. Innerhalb der Phasen liegt der Fokus auf der schrittweisen Realisierung des im Pflichtenheft beschriebenen Soll-Konzepts.

18.1 Phasenübersicht und zeitliche Einordnung

Phase	Zeitraum	Zugeordneter Meilenstein	Inhaltliche Schwerpunkte	Testaktivitäten
1	04.05.2026 – 17.05.2026	M1: Technische Basis und Projektsetup	Einrichtung der Entwicklungsumgebung, Grundstruktur von Frontend und Backend, Vorbereitung zentraler Schnittstellen	Laufende Unit-Tests Meilensteinbezogener Funktionstest
2	18.05.2026 – 31.05.2026	M2: Implementierung der Kernfunktionalitäten	Umsetzung zentraler Nutzerinteraktionen und Kernlogik gemäß Soll-Konzept	Laufende Unit-Tests Meilensteinbezogener Funktionstest
3	01.06.2026 – 14.06.2026	M3: Gamification und Lernformate	Implementierung von Quiz-Funktionalitäten und Gamification-Elementen	Laufende Unit-Tests Meilensteinbezogener Funktionstest
4	15.06.2026 – 28.06.2026	M4: Integration und Systemstabilisierung	Zusammenführung aller Komponenten, Fehlerbehebung, Optimierungen	Laufende Unit-Tests Integrationstest umfassender Ende-zu-Ende-Test
5	29.06.2026 – 06.07.2026	M5: Abschluss und Abnahme	Finalisierung, Dokumentation, Vorbereitung der Abgabe und Präsentation	Abschließender Akzeptanztest des Gesamtsystems

Kapitel I: Testkonzept

Ziel des Testkonzepts ist es, die funktionale Korrektheit, Stabilität und grundlegende Qualität der Anwendung während der Entwicklung sicherzustellen. Aufgrund des Projektcharakters als Proof of Concept (PoC) wird ein bewusst schlankes und pragmatisches Testvorgehen verfolgt, das den Fokus auf wesentliche Risiken legt und gleichzeitig den Entwicklungsaufwand begrenzt.

19 Pair Programming und Code-Review

Zur frühzeitigen Sicherstellung der Codequalität und zur Reduzierung von Fehlern wird im Projekt ergänzend zu den Tests auf Pair Programming sowie Code-Reviews gesetzt. Dieses Vorgehen dient als präventive Qualitätssicherungsmaßnahme und ist fester Bestandteil des Entwicklungsprozesses.

Beim Pair Programming arbeiten zwei Entwickler zeitgleich an derselben Codebasis. Dabei übernimmt eine Person die aktive Implementierung, während die zweite Person den Code kontinuierlich überprüft, fachliche und technische Aspekte hinterfragt und mögliche Fehler frühzeitig erkennt. Dieses Vorgehen unterstützt insbesondere:

- die Einhaltung von Coding-Standards und Architekturvorgaben,
- die frühzeitige Identifikation logischer Fehler,
- den Wissensaustausch innerhalb des Teams,
- sowie eine einheitliche und wartbare Codebasis.

Sofern eine Umsetzung in Form von Pair Programming nicht möglich oder nicht sinnvoll ist, wird der entwickelte Code verpflichtend in einem Code-Review geprüft. Code-Reviews erfolgen durch mindestens ein weiteres Teammitglied und konzentrieren sich insbesondere auf:

- Verständlichkeit und Struktur des Codes,
- korrekte Umsetzung der fachlichen Anforderungen,
- Einhaltung des definierten Berechtigungs- und Sicherheitskonzepts,
- mögliche Fehlerquellen oder Optimierungspotenziale.

Durch die Kombination aus Paarprogrammierung und systematischen Reviews wird sichergestellt, dass die Codequalität bereits während der Entwicklung überprüft wird und nicht ausschließlich von nachgelagerten Testverfahren abhängt.

20 Unit-Tests auf Code-Ebene

Zur Sicherstellung der technischen Qualität werden Unit-Tests für zentrale Komponenten der Anwendung implementiert. Diese Tests prüfen isoliert einzelne Funktionen, Module und Geschäftslogiken, insbesondere im Backend. Ziel ist es, sicherzustellen, dass:

- fachliche Logik korrekt umgesetzt ist,
- Eingaben valide verarbeitet werden,
- definierte Rückgabewerte und Fehlerfälle korrekt behandelt werden,
- Änderungen am Code keine unbeabsichtigten Seiteneffekte verursachen (Regressionen).

Der Schwerpunkt der Unit-Tests liegt auf der Kernlogik (z. B. Orchestrierung, Datenverarbeitung, Zustandslogik), während UI-nahe Aspekte nur eingeschränkt

getestet werden. Die Tests werden entwicklungsbegleitend durchgeführt und dienen primär der Unterstützung der Entwickler während der Implementierungsphase.

21 Meilensteinen Bezogene Funktionstests

Zu definierten Projektmeilensteinen werden **meilensteinbezogene Funktionstests** durchgeführt. Diese Tests prüfen, ob die bis zum jeweiligen Meilenstein umgesetzten Funktionen den Anforderungen des Soll-Konzepts entsprechen.

Die Tests erfolgen aus Nutzersicht und konzentrieren sich auf zentrale Anwendungsfälle, wie beispielsweise:

- Anmeldung und Nutzerverwaltung,
- Nutzung der Kernfunktionalitäten,
- Interaktion mit Lern- und Gamification-Elementen.

Die Durchführung erfolgt anhand einfacher Testfälle oder Checklisten. Ziel ist es, die Funktionsfähigkeit der wichtigsten Komponenten zum jeweiligen Projektstand sicherzustellen.

22 Integrationstests

Zur Überprüfung des Zusammenspiels der einzelnen Systemkomponenten werden **manuelle Integrationstests** durchgeführt. Ziel dieser Tests ist es, sicherzustellen, dass die zuvor einzeln geprüften Module im Gesamtsystem korrekt miteinander interagieren.

Die Integrationstests konzentrieren sich insbesondere auf:

- das Zusammenspiel zwischen Frontend und Backend,
- die korrekte Anbindung und Nutzung der REST-Schnittstellen,
- die Zusammenarbeit von Benutzer-, Rollen- und Berechtigungskomponenten,
- die konsistente Verarbeitung und Weitergabe von Daten zwischen den Systemteilen.

Die Tests werden manuell durch das Projektteam durchgeführt, sobald mehrere funktionale Komponenten miteinander verbunden sind. Dabei werden typische Nutzungsszenarien betrachtet, die mehrere Systemteile gleichzeitig betreffen, beispielsweise die Anmeldung eines Nutzers, der Zugriff auf geschützte Funktionen oder die Nutzung von Lern- und Gamification-Elementen.

23 Abschließender Akzeptanztest

Ergänzend werden einfache manuelle Akzeptanztests durchgeführt. Diese finden jeweils zu festgelegten Meilensteinen im Projektverlauf statt (z. B. nach Abschluss eines Features oder vor einer Abgabe).

Die Akzeptanztests prüfen aus Nutzersicht, ob:

- die umgesetzten Funktionen den definierten Anforderungen entsprechen,
- zentrale Nutzerflüsse (z. B. Login, Chat, Quiz, Dashboard) wie vorgesehen funktionieren,
- keine offensichtlichen Fehler oder Blockaden im Nutzungskontext auftreten.

Die Durchführung erfolgt anhand vordefinierter Testfälle oder Checklisten und wird manuell durch das Projektteam vorgenommen. Eine vollständige oder formale Testabdeckung ist nicht Ziel dieses Ansatzes; vielmehr dienen die Akzeptanztests der funktionalen Absicherung der wichtigsten Features und der Qualitätssicherung vor wichtigen Projektständen.

Kapitel J: **Lizenzen**

Das im Projekt eingesetzte Lizenzmodell ist durch Dennis Schulmeister-Zimolong verbindlich vorgegeben und stellt einen festen Bestandteil der Projektvorgaben dar.

Der im Rahmen des Projekts entwickelte Code wird unter der GNU Affero General Public License (AGPL), Version 3 oder neuer, veröffentlicht. Diese Lizenz gilt verbindlich für den Projektcode sowie für alle integralen Bestandteile des Proof of Concept. Die Umsetzung erfolgt in vollständiger Übereinstimmung mit den aus der AGPL resultierenden technischen und rechtlichen Anforderungen.

Im Zuge der Entwicklung wird sichergestellt, dass ausschließlich Drittkomponenten, Bibliotheken, Laufzeitumgebungen und gegebenenfalls eingesetzte KI-Modelle verwendet werden, deren Lizenzen mit der AGPL kompatibel sind oder eine entsprechende Relizenzierung zulassen. Insbesondere dürfen nur solche Abhängigkeiten integriert werden, die den Copyleft-Bestimmungen der AGPL nicht widersprechen und eine rechtssichere Veröffentlichung des Gesamtwerks unter dieser Lizenz ermöglichen.

Ziel ist es, die vollständige Nutzbarkeit, Weitergabe und Modifizierbarkeit des Projekts im Sinne der vorgegebenen Lizenz sicherzustellen. Daraus ergibt sich die Anforderung, dass das System vollständig mit freien und offenen Softwarekomponenten betrieben werden kann.

Die Einhaltung der Lizenzvorgaben wird während der gesamten Projektlaufzeit berücksichtigt und dokumentiert. Änderungen oder Ergänzungen an den Lizenzanforderungen erfolgen ausschließlich auf Grundlage aktualisierter Vorgaben durch den Dozenten und werden entsprechend im Projektkontext nachvollziehbar festgehalten.

Kapitel K: Risiken und Gegenmaßnahmen

Risiko	Beschreibung	Eintrittswahrscheinlichkeit	Maßnahme
R1: Halluzinationen	Die KI generiert falsche oder irreführende Antworten (NFA03).	Hoch	Einsatz von Retrieval-Augmented Generation (RAG) und strikte Begrenzung der Antwortquellen auf die Vorlesungsmaterialien.
R2: Deskilling	Studierende nutzen die KI unreflektiert und verlieren Kompetenzen (NFA04).	Mittel	Didaktische Einbettung: KI muss Reflexionsfragen stellen und die Antworten des Nutzers hinterfragen.
R3: Open-Source-LLM-Wahl	Das gewählte Open-Source-LLM (NFA01) ist nicht leistungsfähig genug für die didaktischen Anforderungen.	Mittel	Vorab-Evaluierung von mindestens zwei Open-Source-Modellen (z.B. Llama, Mistral) auf Basis der RAG-Performance.
R4: Datenschutz (NFA02)	Speicherung von Chat-Historien (FA02) und personenbezogenen Lernfortschritten.	Mittel	Pseudonymisierung der Daten; strikte Einhaltung der DSGVO; keine Speicherung von Klarnamen im Leaderboard (Annahme).

Kapitel L: Traceability

ID	Anforderung	Konzeptstelle / Feature	Status
FA01	Login/Sign-up	Auth/Profil-Modul, PoC-DoD	MUST
FA02	Chat-Historie	Auth/Profil-Modul, Datenmodell	MUST
FA03	Materialintegration	Content-Integrations-Modul, Datenfluss	MUST
FA04	Proaktive KI-Führung	KI-Core-Modul (Proaktiv), Use Case 4	MUST
FA05	Quizzes	Interaktions-Modul (Quiz), Mockup 2.3	MUST
FA06	Lernfortschrittsanzeige	Dashboard, Lernpfad-Modul	MUST
FA07	Skill Matrix	Dashboard, Gamification-Modul	MUST
FA08	Reaktiver Chat	KI-Core-Modul (Reaktiv), Use Case 2	MUST
FA09	Gamification (Punkte, Streaks)	Gamification-Modul, Use Case 5	SHOULD
FA10	Mindestens 1 Game (PoC)	Interaktions-Modul (Game), PoC-DoD	SHOULD
FA11	Prüfungsplanung	Lernpfad-Modul, Use Case 7	SHOULD
FA12	Wiederholungsfunktion	Dashboard, Lernpfad-Modul	SHOULD
FA13	E-Mail-Benachrichtigungen	Gamification-Modul (Optional)	COULD
FA14	Leaderboard	Gamification-Modul (Optional)	COULD
FA15	Badges	Gamification-Modul (Optional)	COULD
FA16	Quests/Challenges	Gamification-Modul (Optional)	COULD
FA17	Moodle-Anbindung	Ausblick, PoC-Abgrenzung	WON'T
FA18	Multiplayer-Modus	Ausblick, PoC-Abgrenzung	WON'T
NFA01	100% Open Source	Systemkonzept, Risiko R3	MUST

NFA02	Datenschutzkonformität	Risiko R4	MUST
NFA03	Keine Halluzinationen	KI-Core-Modul, Risiko R1	MUST
NFA04	Kein Deskillling	KI-Core-Modul, Risiko R2	MUST
NFA08	Konsistentes UI-Design	UI/UX-Konzept, Stilrichtlinien	MUST

Literaturverzeichnis

- 1EDTech. (2016). *Initial xAPI/Caliper Comparison*. Von Initial xAPI/Caliper Comparison: <https://www.imsglobal.org/initial-xapicaliper-comparison> abgerufen
- 1EdTech. (kein Datum). *Why Platforms and Tools should adopt LTI 1.3*. Von Why Platforms and Tools should adopt LTI 1.3: <https://www.1edtech.org/standards/lti/why-adopt-lti-1p3> abgerufen
- Center, M. N. (18. Januar 2022). Von <https://news.microsoft.com/2022/01/18/microsoft-to-acquire-activision-blizzard-to-bring-the-joy-and-community-of-gaming-to-everyone-across-every-device/> abgerufen
- Dhaliwal, P. (2029). *Structured Prompting Techniques: The complete Guide to XML & JSON*. Von <https://codeconductor.ai/blog/structured-prompting-techniques-xml-json/> abgerufen
- Hecker Consulting. (kein Datum). *ChatGPT wie ein Profi nutzen: Ihre Ultimative Anleitung zu Effektiven Prompts*. Von ChatGPT wie ein Profi nutzen: Ihre Ultimative Anleitung zu Effektiven Prompts: <https://www.hco.de/blog/chatgpt-wie-ein-profi-nutzen-ihre-ultimate-anleitung-zu-effektiven-prompts> abgerufen
- IMS Global. (2019). *Learning Tools Interoperability (LTI)® Advantage Implementation Guide*. Von Learning Tools Interoperability (LTI)® Advantage Implementation Guide: <https://www.imsglobal.org/spec/lti/v1p3/impl> abgerufen
- lernOS. (2024). *Prompts erstellen*. Von Prompts erstellen: <https://ai.lernos.org/de/1-07-prompts-erstellen/> abgerufen
- Mzwri Kovan, T.-S. M. (2025). *The Impact of Prompt Engineering and a Generative AI-Driven Tool on Autonomous Learning: A Case Study*.
- NexAILabs. (2025). *Cracking the Code: JSON or XML for Better Prompts?* Von Cracking the Code: JSON or XML for Better Prompts?: <https://www.nexailabs.com/blog/cracking-the-code-json-or-xml-for-better-prompts> abgerufen
- TurtleDev. (2025). *How to Build a SvelteKit SPA with FastAPI Backend*. Von How to Build a SvelteKit SPA with FastAPI Backend: <https://turtledev.io/blog/how-to-build-sveltekit-spa-with-fastapi-backend> abgerufen

- Xiao, R., Hou, X., Ye, R., Kazemitabaar, M., Diana, N., Luit, M., & Stamper, J. (2025). Improving Student-AI Interaction Through Pedagogical Prompting: An Example in Computer Science Education. <https://arxiv.org/html/2506.19107v2>.
- 1EDTech. (kein Datum). *Learning Tools Interoperability (LTI)®*. Von Learning Tools Interoperability (LTI)®: <https://www.1edtech.org/standards/lti> abgerufen